

Ten Quick Tips to Run Species Distribution Models on High-Performance Computing Clusters

Authors

Co-first: Luca Bütikofer, 1

Co-first: Wim Hordijk, 2

Antoine Adde, 3

Bart Steen, 1

Olivier Brönniman 1,4

Antoine Guisan 1,4

Federico Riva, 2 Email: f.riva@vu.nl ORCID: <https://orcid.org/0000-0002-1724-4293>

1. Department of Ecology and Evolution, University of Lausanne, Biophore building, 1015 Lausanne, Switzerland
2. Department of Environmental Geography, Institute of Environmental Studies, Vrije Universiteit Amsterdam, Amsterdam, the Netherlands
3. Department of Aquatic Ecology, Eawag, Swiss Federal Institute of Aquatic Science and Technology, Dübendorf, Switzerland
4. Institute of Earth Surface Dynamics, University of Lausanne, Geopolis building, 1015 Lausanne, Switzerland

Abstract

Species Distribution Models (SDMs) are increasingly being applied to high-resolution data across large spatial and taxonomic scales, pushing the limits of personal computing. High Performance Computing (HPC) environments offer a powerful solution, but also introduce challenges unfamiliar to many ecologists accustomed to running analyses on personal computers. This article provides practical guidance for overcoming these challenges. Drawing on our experience in fitting SDMs for thousands of species across millions of spatial cells, we outline common difficulties and offer ten hands-on quick tips to overcome them. Our goal is to make HPC-based SDM workflows more accessible, reproducible, sustainable, and efficient for biodiversity researchers.

Introduction

Species distribution models (SDMs) are an important tool in biodiversity science (Guisan, et al., 2013). While historically analyses based on SDMs have been limited by the coarse resolution of available environmental data, the emergence of high-resolution datasets (Jetz, et al., 2019) has shifted the primary limitation from data availability to computational capacity. Meanwhile, growing evidence indicates that high-resolution data can provide important insights on local distribution patterns (Riva, et al., 2024, Bütikofer, et al., 2020) suggesting that the demand for computing resources will dramatically increase in future SDM applications (Adde et al. 2023). This has motivated the development of workflows to fit SDMs on high performance computing (HPC) environments.

HPC environments provide significant parallel computing capacity, which allows thousands of spatial units, iterations, replications, and/or species to be processed simultaneously. Implementing SDM workflows in HPC environments, however, presents significant technical challenges (Adde et al. 2025). Readers seeking a general introduction with setting up an HPC environment can refer to the Simple Rules laid out by Galiwango *et al.* (2025). Here, we share a series of ten Quick Tips specific to the implementation of SDM workflows in HPC environments (Fig. 1).

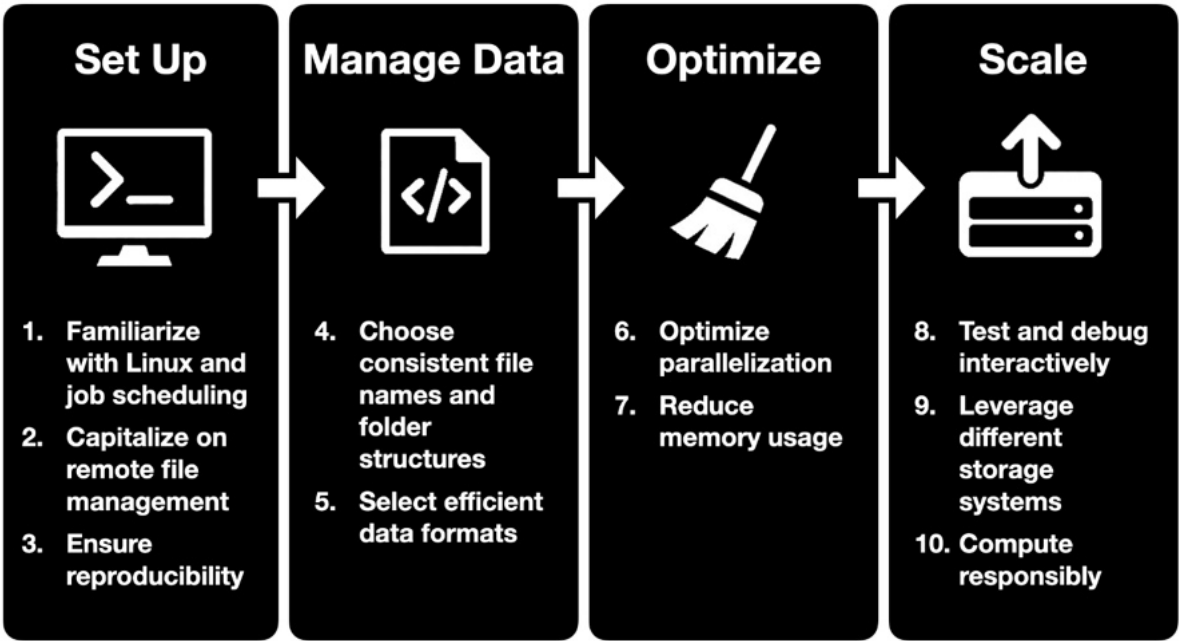


Figure 1: Four areas of focus for effective implementation of SDM workflows in HPC environments, and the ten Quick Tips reported in this manuscript.

Tip 1: Familiarize with Linux and job scheduling

Although fundamental beyond SDMs, a first challenge for new HPC users is that HPC clusters usually run the Linux operating system, rather than MS-Windows or macOS. Linux does include a graphical user interface on personal computers, but HPC clusters often do not provide one. Instead, they offer a *command line interface (shell)*, where actions such as navigating, copying/pasting, writing, saving, and running programs are performed by typing text. Moreover, running SDM workflows (as, e.g., R or Python scripts) is done through a *job scheduler* rather than in an integrated development environment such as RStudio or PyCharm. The job scheduler (e.g., SLURM or PBS) manages and executes job requests according to resource availability and scheduling policies. Each job submission specifies the computational resources required (e.g., memory, CPU cores, GPUs), a maximum walltime, and the program(s) to execute. A simple example of a job submission script is included in the Supplementary Material. Learning basic shell commands and knowing how to write job submission scripts are therefore a first, essential step to running SDMs on HPCs. Alnasir (2021) shares helpful tips to get you started with HPCs, and more detailed guides also exist (Siever, Weber, Figgins, Love, & Robbins, 2005).

Tip 2: Capitalize on remote file management tools

Connecting to an HPC cluster from a Windows personal computer can be challenging for beginners. Users can benefit greatly from third-party software such as MobaXterm (Mobatek, 2025) or equivalent. Such applications combine a command line interface with a graphical file browser, enabling convenient file management on the HPC system (e.g., uploading, downloading, and organizing files). They also include a basic text editor for writing and editing job scripts and other text files directly on the cluster. Note that, since macOS is based on a UNIX-like operating system, the built-in Terminal application can be used to access the HPC environment directly. Also, third-party software such as FileZilla (FileZilla Project, 2024) can be used for file management. In some cases, HPC clusters provide a web interface which allows for file and job management through a graphical user interface running in your web browser.

Tip 3: Ensure software availability and reproducibility

Many software packages to run SDMs rely on libraries such as GDAL or PROJ, which may need to be loaded through the cluster software environment (see Supplementary Material for an example). Make sure that all required software (e.g., R or Python) and supporting libraries are available on the cluster, preferably in their latest version. Otherwise, request your cluster admin to install them, as it might not be possible for individual users to do so. Any missing R or Python packages or modules can often be installed locally (i.e., in the user's home folder). Scripts running on your personal computer will fail unless all needed software is available on the HPC. To improve reproducibility and simplify deployment across computing systems, use standardized

software stacks (optimized system libraries and compilers), isolated virtual environments (project-specific package dependencies), and portable containers (entire modeling environments). This improves portability, scalability and supports best practices for transparent modeling.

Tip 4: Choose consistent file names and folder structures

Data and file management are a critical step in SDM-HPC workflows. File names should be simple, consistent, and machine-readable. Use full species names (e.g., “*Taxus_baccata.tif*”) and avoid spaces, which can break command-line scripts. Furthermore, organize data in separate folders for different data types, while keeping file names identical across folders (e.g., “Occurrences/*Taxus_baccata.csv*”), rather than encoding metadata in file names (e.g., “Occurrences_*Taxus_baccata.csv*”). A folder-based organization improves navigation within large workflows and facilitates matching species across datasets during analysis and visualization. Note that when files are transferred from a personal computer to an HPC file system file permissions (read/write/execute) are not always preserved, which may cause jobs to fail. Check file permissions after upload, and grant the appropriate read, write, and execute permissions as required.

Tip 5: Select efficient data formats

SDM applications often involve large geospatial datasets. Such datasets can cause input/output bottlenecks on HPC clusters. Use compressed formats such as GeoTIFF or NetCDF for both environmental covariate data and output SDM maps. For GeoTIFFs, use lossless compression options. When possible, save output maps as integers rather than floating-point rasters. For continuous suitability values, consider multiplying values by 100 and rounding to the nearest integer to preserve precision (i.e., two decimal places) while reducing file sizes. Additionally, SDMs depend on spatially-explicit data that can be easily mishandled, resulting in errors (Guisan & Thuiller 2005, Anderson *et al.* 2020). Always check coordinate reference systems, extent, and resolution of all geospatial files. Resampling functions are useful for standardizing raster datasets to consistent spatial specifications.

Tip 6: Optimize parallelization

HPC environments allow for parallelization at two levels: (1) within a job, and (2) across jobs. Within a job, multiple processing cores are requested for a single job, which are then allocated to parallel tasks (i.e., processes or threads) within an R or Python script. For example, several models can be calibrated and/or projected in parallel within a single analysis (e.g. an ensemble of models for a given species). With internal parallelization, make sure to request at least as many cores as the number of parallel workers in your script. With parallelization across jobs, the same script is executed many times simultaneously. For example, different species can be analyzed in parallel,

in a classical loop (e.g., with each job executing the same script but with a different species as input). Both methods are useful, and the optimal approach depends on the specifics of your SDM analysis, such as the number of species, models and available resources. Notably, not all SDM approaches are compatible with parallelization (e.g., multi-species joint SDMs based on hyperparameters estimated across the entire species set). Authors should critically evaluate whether parallelization is appropriate, and, if so, which strategy best fits their application.

Tip 7: Reduce memory usage

Increased parallelization comes at the cost of increased memory requirements. It is thus key to find the right trade-off between parallelization and memory usage. One way to reduce memory usage is to avoid unnecessary pre-loading of inputs. For example, in running one of our HPC analyses, we modified a useful function from the ENMeval R package (Kass et al., 2021) to avoid loading entire raster files into memory (see Supplementary Material). Another option is to run fewer parallel processes, but with more memory allocated to each. For instance, on a cluster node with 128 cores and 256 GB of RAM in total, one could run only 64 species in parallel rather than 128, increasing the available memory from ~2 GB to ~4 GB per process. Furthermore, for memory-intensive steps (e.g., model projections), memory usage can be reduced by partitioning the data into subsets. For example, we divided SDM projection areas into “tiles” that were processed sequentially—or with limited parallelism—and then merged back into a single output map (see Supplementary Material). Finally, when using R, memory usage may sometimes be reduced through a more aggressive garbage collection strategy. This can be done by placing a file named “.Renvi ron” in your home directory (or the directory from where the compute jobs are submitted), with the following line “R_GC_MEM_GROW=0”. This file is automatically read by R at startup. In our applications, this simple modification reduced memory usage by up to 25%.

Tip 8: Test and debug interactively

Although jobs submitted to the scheduler run non-interactively (“in the background”), most HPC clusters also allow users to connect interactively for testing and debugging purposes. In integrated development environments, such as RStudio or PyCharm, users can typically send individual commands or code blocks directly to a console or terminal. When connected to an HPC cluster through an SSH session, similar approaches can be used to test and debug scripts directly in the HPC environment. Once an SDM script runs without errors, test it in an interactive session using a small pilot analysis (e.g., a handful of species) to estimate overall runtime and memory requirements. This will help determine the appropriate resources needed for the full analysis. Job schedulers offer commands to track resource consumption, and some clusters provide web-based interfaces for launching interactive sessions and monitoring job status, memory usage, and CPU utilization.

Tip 9: Leverage different storage systems

Since SDMs often run on large datasets, reading and writing files can be significant bottlenecks. HPC environments generally provide multiple storage systems, often including: (1) a temporary high-performance storage system for active computations, and (2) one or more long-term storage systems intended for persistent data storage. The long-term storage system (usually “home” or “work” directory) is typically permanent and regularly backed up, but with slower file access from the cluster CPUs. The short-term storage system (usually “scratch” directory) tends to have faster read/write speed and larger capacity, but it is not automatically backed up and is regularly purged. Store SDM code and input files permanently on the long-term storage system (e.g., scripts, environmental variables, biodiversity data). For active jobs, input files can be temporarily transferred to the scratch system, where intermediate and final results (e.g., model objects) are also saved. Once all jobs are complete, results that need to be retained or further processed can be transferred back to the long-term storage system. Users should verify whether their HPC cluster follows from these common storage practices, as storage configurations and policies may vary among systems.

Tip 10: Compute responsibly

Several of the tips provided above serve a fundamental purpose: reducing carbon emissions associated with HPC usage. Two major causes of energy waste and excessive CO₂ emissions are memory overallocation (Lannelongue et al. 2021) and premature cancellation of cluster jobs. Both can often be avoided by reducing memory use, running smaller tests in interactive sessions first, and using resource monitoring software (see tips 7-9). Finally, if Large Language Models (LLMs) have been installed on the HPC cluster, they can be used as local coding assistants. Local LLMs may reduce data transfer requirements and reliance on external cloud services (Ding et al. 2025).

Conclusions

This Quick Tips article was designed to help researchers interested in SDMs capitalize on the potential of HPC clusters, building on previous articles designed to more broadly improve computing practices (Alnasir 2021, Galiwango *et al.* 2025). SDMs fitted to millions of pixels and thousands of species will be necessary to answer cutting-edge questions in biodiversity science. Such models are increasingly available to ecologists thanks to advances in data availability and computing infrastructure—especially HPC environments. However, harnessing HPC resources requires technical skills that are not typically required for running SDMs on a personal computer. If your institution provides access to HPC resources, it likely also provides courses and tutorials for learning how to use them effectively. In our experience, investing time in learning the specifics of your HPC environment can save you days of debugging. We have attempted

here to cover the basics for implementing SDMs in HPCs (Fig. 2), but acknowledge that robust workflows can benefit from more detailed information on each of the mentioned steps (Fig. 1).

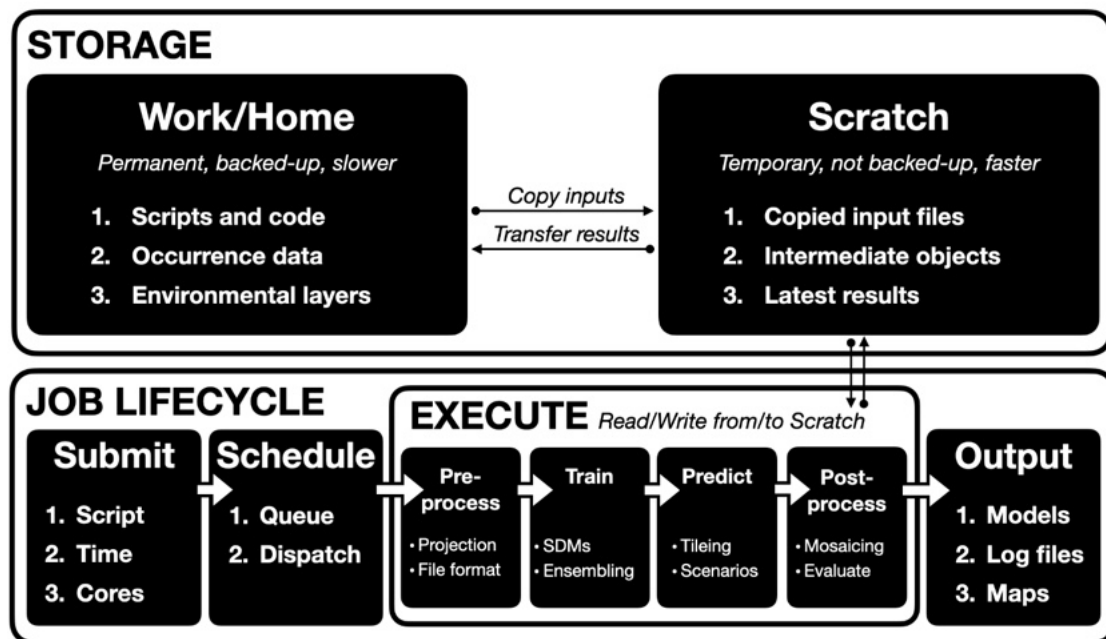


Figure 2: Operational workflow for running SDMs on an HPC cluster. Storage (top) is divided into long-term storage (Work/Home) and fast temporary storage (Scratch). Input files are copied to Scratch before execution and results are transferred back to Work/Home upon completion. The job lifecycle (bottom) proceeds from submission (users provide script, execution time, and number of cores), through scheduling (queuing and dispatching by the job scheduler), to execution (where the core SDM pipeline runs), ultimately generating the different outcomes that conservation scientists can use from species distribution models workflows.

Ultimately, many common issues associated with HPC use can be addressed through relatively straightforward practices, which we summarized in this paper. With preparation and awareness, ecologists can unlock the full potential of HPC environments to accelerate SDM workflows and scale up biodiversity assessments. Such analyses should be implemented efficiently and with attention to their computational footprint, reducing our contribution to the biodiversity declines we are trying to understand with SDMs. We hope this paper will help researchers overcome the challenges we faced as we developed SDM workflows for HPC environments, promoting the adoption of best practices in computational biology.

Acknowledgements

We acknowledge funding support from Swiss Re and enabling partners AXA Research Fund, WWF, Swiss Re Institute and as a service contributor, EY for the "Modelling

229 biodiversity and ecosystem service loss to advance resilience" project, the Swiss
230 Confederation through the [Valpar.ch](https://valpar.ch) project, and the European Commission through
231 the COUSIN Horizon project.

232 References

- 233 Adde, A., Rey, P.L., Külling, N. *et al.* *SDMapCH*: a Comprehensive database of >7,500
234 modelled species habitat suitability maps for Switzerland. *Sci Data* 12, 1752 (2025).
235 <https://doi.org/10.1038/s41597-025-06037-x>
- 236 Adde, A., Külling, N., Rey, P.-L., Fopp, F., Brun, P., Broenniman, O., Lehmann, A.,
237 Petitpierre, B., Zimmermann, N. E., Pellissier, L., Altermatt, F. & Guisan, A. (2024)
238 Projecting Untruncated Climate Change Effects on Species' Climate Suitability: Insights
239 From an Alpine Country. *Global Change Biology*, 30.
- 240 Adde, A., Rey, P. L., Brun, P., Kuelling, N., Fopp, F., Altermatt, F., Broennimann, O.,
241 Lehmann, A., Petitpierre, B., Zimmermann, N. E., Pellissier, L. & Guisan, A. (2023) N-
242 SDM : a high-performance computing pipeline for Nested Species Distribution
243 Modelling. *Ecography*, 2023(6):e06540.
- 244 Alnasir, J.J. (2021) Fifteen quick tips for success with HPC, i.e., responsibly BASHing
245 that Linux cluster. *PLoS Comput Biol* 17(8): e1009207.
- 246 Anderson, R. P., Araujo, M. B., Guisan, A., Lobo, J. M., Martinez-Meyer, E., Peterson, A.
247 T. & Soberon, J. M. (2020) Optimizing biodiversity informatics to improve information
248 flow, data quality, and utility for science and society. *Frontiers of Biogeography*, 12.
- 249 Bütikofer L, Anderson K, Bebbier D P, Bennie J J, Early R I and Maclean I M D 2020 The
250 problem of scale in predicting biological responses to climate *Global Change Biology* 26
251 6657–66
- 252 Ding, Z., Wang, J., Song, Y., Zheng, X., He, G., Chen, X., Zhang, T., Lee, W.-J., & Song, J.
253 (2025). Tracking the carbon footprint of global generative artificial intelligence. *The*
254 *Innovation*, 6(5), 100866.
- 255 FileZilla. FileZilla Project, Version 3.66, 2024, <https://filezilla-project.org>.
- 256 Galiwango R., Whalen C.J., Kebirungi G., Atwine M.T., Kimera R., Ssekagiri A., et al.
257 (2025) Ten simple rules for building and maintaining sustainable high-performance
258 computing infrastructure for research in resource-limited settings. *PLoS Comput Biol*
259 21(9): e1013481. <https://doi.org/10.1371/journal.pcbi.1013481>
- 260 Guisan, A., Tingley, R., Baumgartner, J. B., Naujokaitis-Lewis, I., Sutcliffe, P. R., Tulloch,
261 A. I., . . . Buckley, Y. M. (2013). Predicting species distributions for conservation
262 decisions. *Ecology Letters*, 16(12), 1424-1435.

263 Guisan, Antoine, and Wilfried Thuiller. "Predicting species distribution: offering more
264 than simple habitat models." *Ecology letters* 8.9 (2005): 993-1009.

265 Jetz, W., McGeoch, M. A., Guralnick, R., Ferrier, S., Beck, J., Costello, M. J., . . . Turak, E.
266 (2019). Essential biodiversity variables for mapping and monitoring species
267 populations. *Nature Ecology & Evolution*, 3, 539-551.

268 Kass, J. M., Smith, A. B., Warren, D. L., Vignali, S., Schmitt, S., Aiello-Lammens, M. E.,
269 Arlé, E., Barbosa, A. M., Broennimann, O., Cobos, M. E., Gueguen, M., Guisan, A.,
270 Merow, C., Naimi, B., Nobis, M. P., Ondo, I., Osorio-Olvera, L., Owens, H. L., Pinilla-
271 Buitrago, G. E., Sanchez-Tapia, A., Thuiller, W., Valavi, R., Velazco, S. J. E., Zizka, A. &
272 Zurell, D. (2024) Achieving higher standards in species distribution modeling by
273 leveraging the diversity of available software. *Ecography*.

274 Kass, J. M., Muscarella, R., Galante, P. J., Bohl, C. L., Pinilla-Buitrago, G. E., Boria, R. A.,
275 . . . Anderson, R. P. (2021). ENMeval 2.0: Redesigned for customizable and reproducible
276 modeling of species' niches and distributions. *Methods in Ecology and Evolution*, 12(9),
277 1602-1608.

278 Kays, R., McShea, W. J. & Wikelski, M. (2020) Born-digital biodiversity data: Millions and
279 billions. *Diversity and Distributions*, 26, 644-648.

280 Lannelongue, L., Grealey, J., & Inouye, M. (2021). Green Algorithms: Quantifying the
281 Carbon Footprint of Computation. *Advanced Science*, 8(12), 2100707.
282 <https://doi.org/10.1002/advs.202100707>

283 Mobatek. (2025). MobaXterm. Retrieved from <https://mobaxterm.mobatek.net>

284 Phillips, S. J., Anderson, R. P., Dudik, M., Schapire, R. E., & Blair, M. E. (2017). Opening
285 the black box: an open-source release of Maxent. *Ecography*, 40(7), 887-893.

286 Riva, F., Martin, C. J., Acedo, C. G., Bellon, E. N., Keil, P., Moran-Ordóñez, A., . . .
287 Guisan, A. (2024). Incorporating effects of habitat patches into species distribution
288 models. *Journal of Ecology*, 112(10), 2162-2182.

289 Siever, E., Weber, A., Figgins, S., Love, R., & Robbins, A. (2005). *Linux in a Nutshell* (Fifth
290 ed.). O'Reilly.